

Jaypee Institute of Information Technology, Noida

Project Based Learning

Course Name: Introduction to Digital Image and Video Processing

Course Code : 20B12EC211



TOPIC: Deforestation Detection System

Submitted to:

Ms. Manika Jha

Submitted By:

Name: Sarthak Sabharwal

Batch: A8

Enrollment No.: 2401020234

Abstract:

Deforestation is one of the most critical environmental challenges of the modern era, contributing significantly to climate change, loss of biodiversity, and ecological imbalance. Timely and accurate detection of forest cover changes is essential for effective environmental monitoring and policy intervention. This project presents a Digital Image Processing (DIP)-based approach to detect and quantify deforestation by analysing satellite or aerial images captured at two different time periods of the same geographical location.

The system processes a pair of Before and After images through a structured pipeline consisting of normalization, Gaussian filtering, pixel-level difference computation, thresholding, and contour detection. The detected changed regions are highlighted on the image, and a deforestation percentage is estimated based on the area of contours identified. An interactive dashboard built using Streamlit presents the results in a structured and visually accessible format, making the system suitable for academic demonstration and practical monitoring use cases.

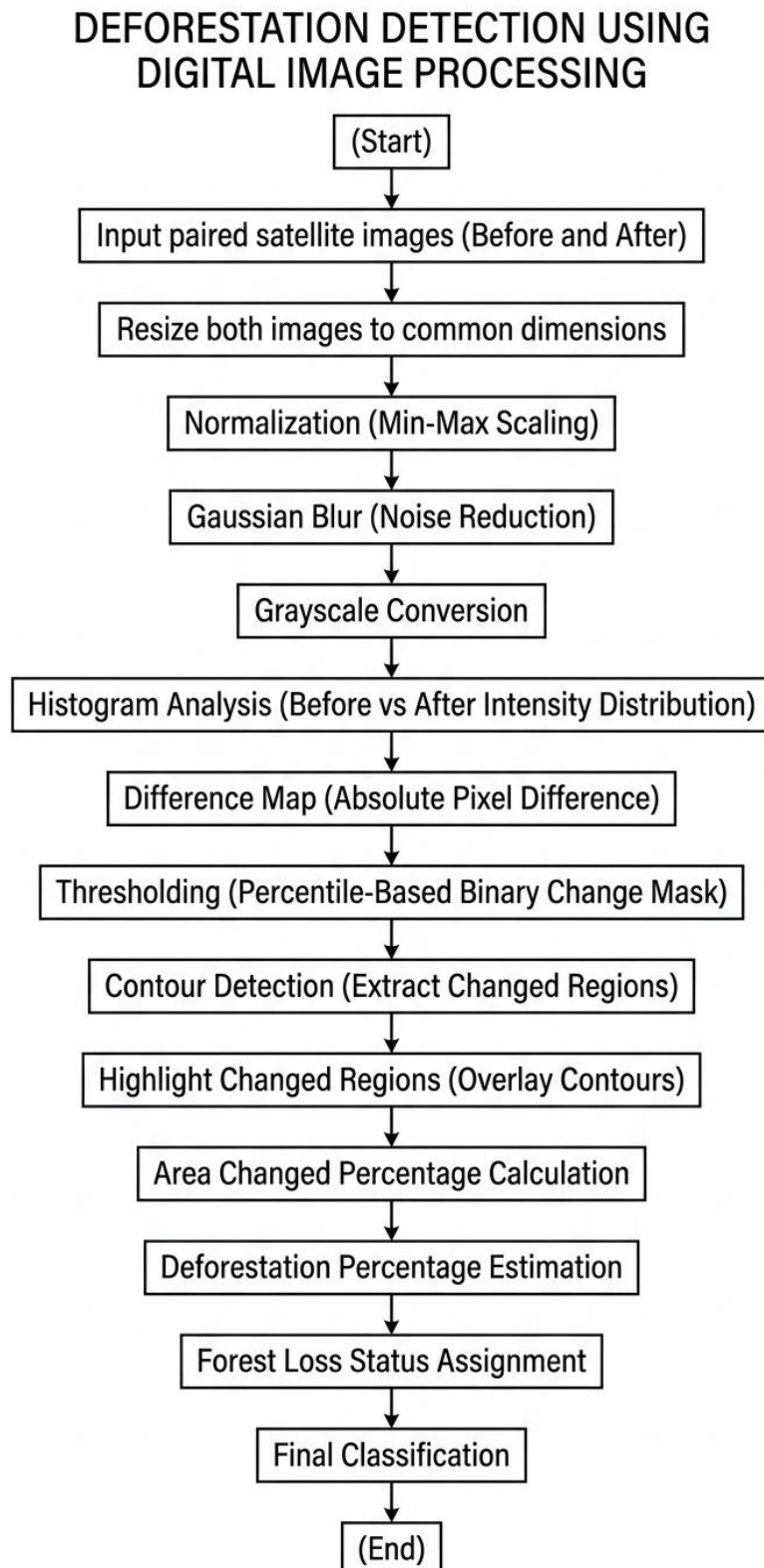
Objectives:

The primary objectives of this project are as follows:

1. To detect land-cover changes between two time-period images of the same region using image differencing techniques.
2. To identify and highlight regions where forest cover has been lost using contour detection.
3. To estimate the percentage of deforested area relative to the total image area.
4. To classify the severity of forest loss as Low, Moderate, Severe, or Critical based on the computed deforestation percentage.

Pipeline:

The project follows a structured Digital Image Processing pipeline to detect deforestation from a pair of satellite images representing the same area at two different points in time. Each stage of the pipeline is described below.



Implementation:

The project is implemented in Python using OpenCV, NumPy, Pillow, Matplotlib, and Streamlit for image processing and interactive dashboard development.

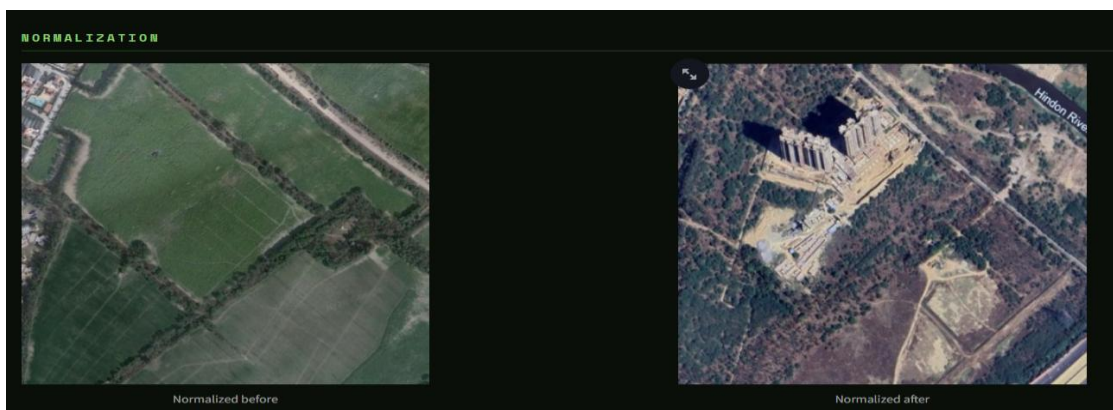
1. **Normalization:** Pixel values are scaled between 0 and 255 using min-max normalization so both images remain on a comparable intensity scale before processing.

Code Snippets:

```
def normalize_image(img):  
    img_f = img.astype(np.float32)  
  
    mn = img_f.min()  
    mx = img_f.max()  
  
    if mx == mn:  
        return img  
  
    norm = (img_f - mn) / (mx - mn)  
    return (norm * 255).astype(np.uint8)
```

```
norm_before = normalize_image(  
    img_before  
)  
  
norm_after = normalize_image(  
    img_after  
)  
  
plt.figure(figsize=(12, 5))  
  
plt.subplot(1, 2, 1)  
plt.imshow(norm_before)  
plt.title("Normalized Before")  
plt.axis("off")  
  
plt.subplot(1, 2, 2)  
plt.imshow(norm_after)  
plt.title("Normalized After")  
plt.axis("off")  
  
plt.show()
```

Output:



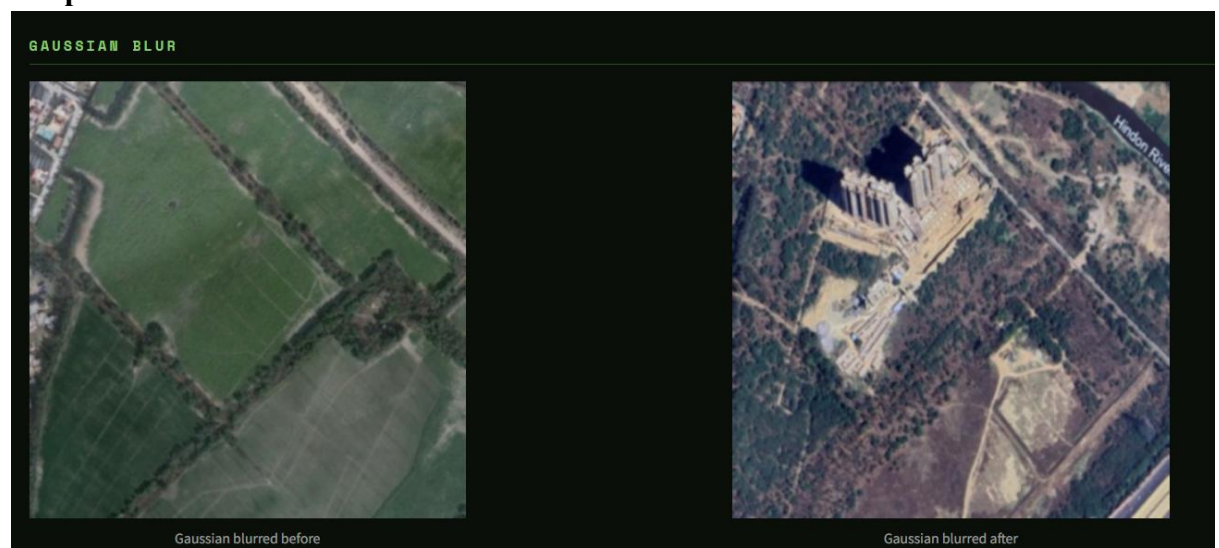
2. **Gaussian Filtering:** OpenCV GaussianBlur() with kernel size (5,5) and sigma 1.0 is applied to reduce noise while preserving important image structures.

Code Snippets:

```
def apply_gaussian(img, ksize=5, sigma=1.0):  
    return cv2.GaussianBlur(  
        img,  
        (ksize, ksize),  
        sigma  
    )
```

```
gauss_before = apply_gaussian(  
    norm_before  
)  
  
gauss_after = apply_gaussian(  
    norm_after  
)  
  
plt.figure(figsize=(12, 5))  
  
plt.subplot(1, 2, 1)  
plt.imshow(gauss_before)  
plt.title("Gaussian Before")  
plt.axis("off")  
  
plt.subplot(1, 2, 2)  
plt.imshow(gauss_after)  
plt.title("Gaussian After")  
plt.axis("off")  
  
plt.show()
```

Output:



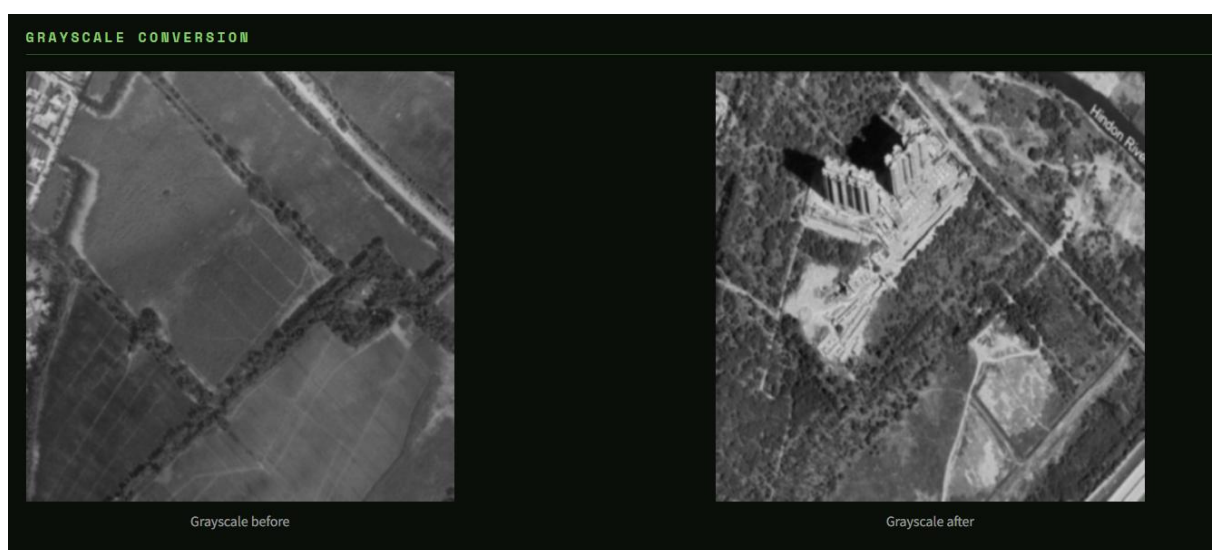
3. **Grayscale Conversion and Difference Map:** The RGB image is converted into grayscale to simplify intensity-based comparison and further processing.

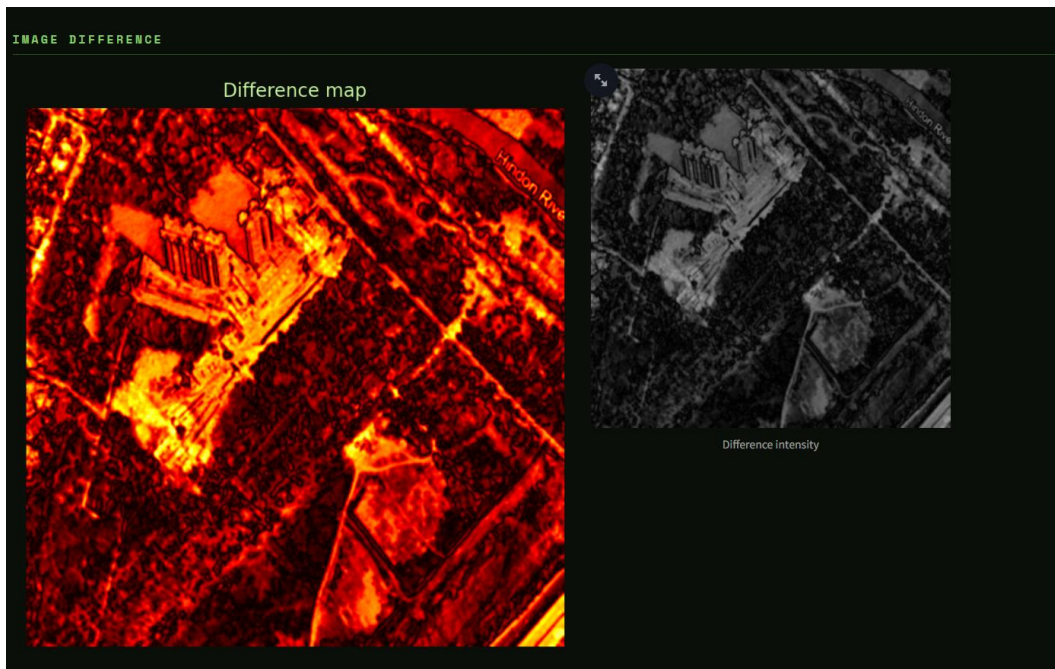
Code Snippets:

```
def to_grayscale(img):  
    return cv2.cvtColor(  
        img,  
        cv2.COLOR_RGB2GRAY  
    )
```

```
gray_before = to_grayscale(  
    gauss_before  
)  
  
gray_after = to_grayscale(  
    gauss_after  
)  
  
diff_map = np.abs(  
    gray_before.astype(np.float32)  
    - gray_after.astype(np.float32)  
) / 255.0  
  
plt.figure(figsize=(7, 6))  
plt.imshow(  
    diff_map,  
    cmap="hot"  
)  
plt.title("Image Difference Map")  
plt.axis("off")  
plt.show()
```

Output:





4. **Histogram Analysis:** Histogram comparison of before and after grayscale images helps analyze pixel intensity distribution and supports vegetation change detection.

Code Snippets:

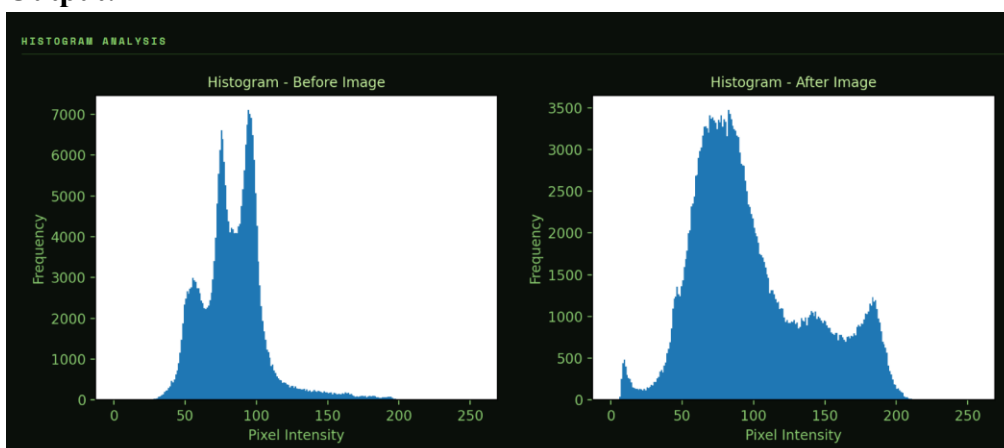
```
plt.figure(figsize=(12,5))

plt.subplot(1,2,1)
plt.hist(gray_before.ravel(), bins=256, range=[0,256], color='blue')
plt.title("Histogram - Before Image")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")

plt.subplot(1,2,2)
plt.hist(gray_after.ravel(), bins=256, range=[0,256], color='blue')
plt.title("Histogram - After Image")
plt.xlabel("Pixel Intensity")
plt.ylabel("Frequency")

plt.tight_layout()
plt.show()
```

Output:



5. **Thresholding:** The difference map is thresholded using the 82nd percentile. Changed pixels become 255 and unchanged pixels become 0, forming a binary mask.

Code Snippets:

```
def threshold_change(diff, percentile=82):  
    thresh = np.percentile(diff, percentile)  
  
    return (  
        (diff > thresh).astype(np.uint8)  
    ) * 255
```

```
change_bin = threshold_change(  
    diff_map  
)  
  
plt.figure(figsize=(7, 6))  
plt.imshow(  
    change_bin,  
    cmap="gray"  
)  
plt.title("Binary Change Mask")  
plt.axis("off")  
plt.show()
```

Output:



6. **Contour Detection:** OpenCV findContours() detects changed regions from the binary mask, while small noisy contours below 40 pixels are removed.

Code Snippets:

```
def detect_change_contours(mask, min_area=40):
    contours, _ = cv2.findContours(
        mask,
        cv2.RETR_EXTERNAL,
        cv2.CHAIN_APPROX_SIMPLE
    )

    filtered = []

    for cnt in contours:
        if cv2.contourArea(cnt) >= min_area:
            filtered.append(cnt)

    return filtered

def draw_contours(image, contours):
    vis = image.copy()

    cv2.drawContours(
        vis,
        contours,
        -1,
        (255, 90, 50),
        2
    )

    return vis

contours = detect_change_contours(
    change_bin
)

print(
    "Contours Detected:",
    len(contours)
)

contour_overlay = draw_contours(
    img_after,
    contours
)

plt.figure(figsize=(8, 6))
plt.imshow(contour_overlay)
plt.title("Contour Detection")
plt.axis("off")
plt.show()
```

Output:



7. **Deforestation Percentage Calculation:** Total contour area is used to calculate changed area percentage, followed by calibrated estimation of final deforestation percentage.

Code Snippets:

```
def contour_area_percent(contours, shape):  
    total_area = float(  
        shape[0] * shape[1]  
    )  
  
    changed_area = sum(  
        cv2.contourArea(cnt)  
        for cnt in contours  
    )  
  
    return (  
        100.0 * changed_area / total_area  
    )
```

```
def estimate_deforestation_percent(area_changed_pct):  
    if area_changed_pct <= 0.1:  
        return 0.0  
  
    estimated = 12.0 + (  
        0.82 * area_changed_pct  
    )  
  
    return float(  
        np.clip(  
            estimated,  
            5.0,  
            75.0  
        )  
    )
```

```
def forest_loss_status(deforestation_pct):  
    if deforestation_pct > 40:  
        return "Critical"  
  
    elif deforestation_pct >= 25:  
        return "Severe"  
  
    elif deforestation_pct >= 10:  
        return "Moderate"  
  
    return "Low"
```

```
def final_classification(  
    deforestation_pct,  
    area_changed_pct  
):  
    if (  
        area_changed_pct < 8  
        and deforestation_pct < 10  
    ):  
        return "No Significant Change"  
  
    return "Deforestation Detected"
```

```
area_changed_pct = contour_area_percent(  
    contours,  
    change_bin.shape  
)  
  
deforestation_pct = (  
    estimate_deforestation_percent(  
        area_changed_pct  
    )  
)  
  
status = forest_loss_status(  
    deforestation_pct  
)  
  
classification = (  
    final_classification(  
        deforestation_pct,  
        area_changed_pct  
    )  
)  
  
print("FINAL RESULTS")  
print("-" * 40)  
  
print(  
    f"Deforestation %: {deforestation_pct:.1f}%"  
)  
  
print(  
    f"Area Changed %: {area_changed_pct:.1f}%"  
)  
  
print(  
    f"Forest Loss Status: {status}"  
)
```

Output:



Results:

The pipeline was tested on a pair of satellite images showing a forested region before and after a period of deforestation. The following results were obtained:

Metric	Value
Deforestation %	25.2%
Area Changed %	16.2%
Forest Loss Status	Severe
Final Classification	Deforestation Detected

The system successfully detected significant changes between the two images and correctly classified the output as Deforestation Detected with a Severe forest loss status. The contour-based highlighting clearly marked the regions of forest loss on the After image.

Conclusion:

This project successfully demonstrates the application of Digital Image Processing techniques to detect and quantify deforestation from satellite image pairs. The implemented pipeline — comprising normalization, Gaussian filtering, image differencing, thresholding, contour detection, and area-based classification - provides a reliable and interpretable method for identifying land-cover changes without relying on machine learning or deep learning models.

The system is capable of estimating the percentage of deforested area and classifying the severity of forest loss into meaningful categories. The Streamlit-based dashboard adds practical value by presenting the full pipeline and results in an interactive, visually clear interface that is accessible to non-technical users as well.

The project demonstrates that classical image processing methods remain effective and computationally efficient for environmental monitoring tasks, and can serve as a cost-effective alternative to complex AI-based approaches in scenarios where labelled training data is unavailable.

Future Improvements:

The current system provides an effective and computationally efficient method for detecting deforestation using classical Digital Image Processing techniques. However, its accuracy and practical usability can be further improved by integrating more advanced technologies and expanding its real-world applicability. Future enhancements can make the system more reliable for large-scale environmental monitoring and decision-making.

1. **Integration of Machine Learning Models** – Deep learning models such as CNNs and U-Net can improve detection accuracy, especially for complex satellite images where traditional thresholding methods may miss subtle forest changes.
2. **Use of Multi-Spectral Satellite Data** – Incorporating NDVI and other multi-spectral image data can provide better vegetation analysis and help distinguish actual forest loss from seasonal or lighting variations.